

Banc d'évaluation et d'optimisation de LLM pour le support IT (contexte BibOps)

Contexte

Les grands modèles de langage (LLM) sont aujourd'hui capables d'assister efficacement les équipes de support IT sur des tâches variées : classification de tickets, aide au diagnostic d'incidents, synthèse de journaux d'événements, proposition de réponses aux utilisateurs, etc.

Au-delà de l'usage "brut" d'un LLM, on voit apparaître de plus en plus de systèmes dits « agentiques », dans lesquels un ou plusieurs LLM sont orchestrés comme des agents spécialisés (par exemple : un agent pour analyser les logs, un agent pour rédiger une réponse, un agent pour vérifier la cohérence) qui collaborent pour résoudre une tâche de support IT.

Dans un contexte industriel, les modèles les plus performants posent toutefois plusieurs problèmes : coût d'inférence élevé, latence, empreinte carbone, dépendance forte à un fournisseur. Il devient donc stratégique de savoir dans quelle mesure des modèles plus petits (hébergés en SaaS ou auto-hébergés), éventuellement organisés en systèmes d'agents, peuvent fournir une qualité de réponse jugée acceptable pour le support IT.

Chez Michelin, le projet BibOps vise à explorer l'usage de l'IA pour le support IT et l'observabilité. Le présent projet académique ne manipule pas directement BibOps ni le SI Michelin : il consiste à construire un banc d'évaluation générique permettant de comparer différents LLM sur des cas d'usage de support IT, et à explorer des stratégies (dont des approches agentiques simples) pour obtenir des résultats satisfaisants avec des modèles de plus petite taille.

Contenu du projet

L'objectif global est de concevoir et implémenter une méthodologie reproductible pour évaluer et optimiser des LLM dans des scénarios de support IT. Le projet pourra se structurer en plusieurs étapes :

1. Étude du contexte et des cas d'usage de support IT
 - Revue de littérature et de ressources publiques sur l'usage des LLM pour le support IT (classification de tickets, FAQ, aide au diagnostic, etc.), et sur les approches « agentiques » (agents spécialisés, boucles perception-raisonnement-action simples).
 - Sélection de quelques cas d'usage représentatifs (par exemple : proposer une réponse à un ticket utilisateur, résumer un échange technique, suggérer des pistes de résolution à partir d'un court descriptif).
 - Définition d'un ou plusieurs « modèles de référence » (LLM performants) qui serviront de baseline qualitative.

2. Conception d'un jeu de tests et de critères d'évaluation

- Construire un dataset de scénarios de support IT (tickets ou demandes synthétiques ou anonymisés, issues de sources publiques ou générées).
- Définir des critères d'évaluation : pertinence de la réponse, complétude, clarté, robustesse, temps de réponse, coût estimé.
- Proposer une méthodologie d'évaluation combinant métriques automatiques (scores, heuristiques) et éventuellement un protocole d'évaluation humaine.

3. Implémentation d'un banc de benchmark multi-modèles

- Développer en Python un banc de test permettant d'appeler plusieurs LLM (API publiques ou modèles open-source) via une interface unifiée.
- Exécuter automatiquement le jeu de tests sur chaque modèle, collecter les sorties et calculer les métriques définies.
- Générer des rapports de comparaison (tableaux, visualisations) mettant en évidence les points forts/faiblesses de chaque modèle.
- Optionnel : prévoir que certains scénarios puissent être résolus soit par un LLM unique, soit par une petite composition d'agents (par exemple deux appels successifs : analyse puis reformulation), afin de préparer la comparaison entre architectures « monolithiques » et « agentiques simples ».

4. Stratégies d'optimisation pour obtenir de bons résultats avec des modèles plus petits

- Expérimenter des techniques de prompt engineering (gabarits, contraintes explicites, structuration de la sortie).
- Tester des stratégies de raisonnement amélioré (demander au modèle de détailler ses étapes, auto-vérification simple, etc.).
- Mettre en place un schéma simple d'augmentation contextuelle (RAG léger) en injectant, pour chaque scénario, une petite base de connaissances pertinente (FAQ, procédures, documentation synthétique).
- Explorer des règles de post-traitement (vérification de champs obligatoires, formats attendus) pour améliorer la fiabilité perçue.
- Optionnel : prototyper une mini-architecture agentique (par exemple, un agent « analyse » qui prépare les éléments techniques et un agent « réponse » qui rédige le message utilisateur) et comparer ses performances avec celles d'un LLM utilisé seul.

5. Synthèse et perspectives d'intégration dans une plateforme type BibOps

- Synthétiser les résultats sous la forme de recommandations pour le choix de LLM selon les contraintes coût/latence/qualité.
- Discuter comment le banc d'évaluation pourrait être intégré ou adapté dans une plateforme interne comme BibOps, aussi bien pour tester des modèles que pour comparer des architectures (LLM unique vs petits systèmes d'agents) sur des cas d'usage de support IT.

- Documenter clairement le code produit et la méthodologie, de façon à ce qu'elle soit réutilisable pour d'autres jeux de cas d'usage.

Le périmètre précis (nombre de modèles, taille du dataset, profondeur des expériences, degré de sophistication des agents) sera ajusté en fonction du temps disponible, mais l'esprit du projet reste : mesurer, comparer, puis optimiser en vue de rationaliser le choix de LLM et, à terme, d'architectures agentiques pour le support IT.

Compétences requises

- Python (indispensable) : scripts, manipulation de données, appels d'API.
- Stratégie de test / ingénierie des tests : définition de cas de test, métriques, protocoles de benchmark.
- Connaissances de base en modèles de langage (LLM), prompts, génération de texte.
- Confort avec Git et un environnement de développement collaboratif.

Une curiosité pour les architectures à base d'agents (agents LLM, systèmes multi-agents simples) constitue un plus, mais ne constitue pas un prérequis bloquant.

Des connaissances en apprentissage automatique, en MLOps ou en outils de support IT (systèmes de tickets, FAQ) sont également appréciées, sans être indispensables au démarrage.

Encadrement

Le projet est encadré par Émilien Mottet, IT Expert, SRE chez Michelin et travaillant sur le projet BibOps d'assistance IA au support IT.

Contact : emilien.mottet@michelin.com